

Practical Guide – Building a Single Agent with Tools

This guide walks you step by step. At each step, you'll **understand what to do**, and only the **instructor's solution** will show the full code.

Step 1. Setup Your Environment

- Install required Python libraries:
 - `langchain`, `langchain_openai`, `langchain_community`, `langgraph`, `python-dotenv`
- Save your OpenAI API key in a `.env` file.

Why?

This ensures you have all the tools and keys needed for the agent to run. LangChain needs the OpenAI key for its LLM.

Step 2. Initialize the Chat Model

- Use `ChatOpenAI` with a lightweight model (e.g., `gpt-4o-mini`).
- Add a **system prompt** to tell the agent what its role is. Example: *"You are a travel assistant who helps with weather, routes, and clothing."*

Why?

The agent needs a base personality and purpose to decide how to use the tools.

Step 3. Define the Tools

We will simulate three tools. You'll later see their code in the instructor version.

1. **Weather API tool**
 - Input: a location (e.g., "Paris")
 - Output: a random weather condition (e.g., "sunny", "rainy").
2. **Route Planner API tool**
 - Input: destination + weather condition
 - Output: best travel route (e.g., *"take the subway if raining"*).

3. Clothing Recommendation API tool

- Input: weather condition
- Output: clothing advice (e.g., “*wear light clothes if hot*”).

Why?

Each tool does one specific job. The agent will combine them depending on the query.

Step 4. Connect Tools to the Agent

- Gather all tools into a list.
- Use `create_react_agent()` to bind the tools to the chat model.
- Pass in the system prompt so the agent knows its role.

Why?

This is what makes the agent capable of “deciding” when to use which tool.

Step 5. Run Your First Query

- Try a question that asks about the **weather** in a city.
- See how the agent calls the **weather tool** to get the answer.

Why?

This confirms the agent can call tools when needed.

Step 6. Try More Queries

Here are test queries. Each one is designed to trigger **different tool combinations**.

Tool Testing Queries

1. **Weather only**
 - “What is the weather like in Paris today?”
2. **Weather + Clothing**
 - “I am traveling to Tokyo tomorrow. Can you tell me the weather and what I should wear?”
3. **Weather + Route**
 - “If I go to London today, what’s the weather and what route should I take?”
4. **Route + Clothing (weather given inside the question)**

- “It’s raining in Singapore. How should I travel to the Marina Bay area, and what should I wear?”
5. **Weather + Route + Clothing**
- “I’m planning a trip to New York tomorrow. Please tell me the weather, the best route to Central Park, and what clothing to pack.”

Why?

These queries test all tools, separately and in combinations. You can observe how the agent decides what to use.



Instructor Reference Code

Below is the **full code** (already explained earlier), but with comments showing how each tool works. Learners can compare this with their own results.

```
"""
Instructor Solution: LangChain Single Agent with Tools
This script shows how to build a simple agent using LangChain's ReAct agent
with three simulated APIs (weather, route, clothing).
"""

import os, random
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage
from langchain.tools import tool
from langgraph.prebuilt import create_react_agent

# Load API key
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

# -----
# Define Tools
# -----
@tool
def weather_api(location: str) -> str:
    """Simulated Weather API: returns random weather for the given
    location."""
    conditions = ["sunny", "rainy", "stormy", "cloudy", "hot", "cold",
    "windy"]
    weather = random.choice(conditions)
    return f"The weather in {location} is {weather}."

@tool
def route_api(destination: str, weather: str) -> str:
    """Simulated Route Planner: suggests route based on weather."""
    if "rain" in weather or "storm" in weather:
```

```
        return f"Best route to {destination}: take public transport, as
weather is {weather}."
    else:
        return f"Best route to {destination}: enjoy a scenic walk, since
weather is {weather}."

@tool
def clothing_api(weather: str) -> str:
    """Simulated Clothing API: recommends clothing based on weather."""
    if "sunny" in weather or "hot" in weather:
        return "Wear light clothes, sunglasses, and a hat."
    elif "rain" in weather or "storm" in weather:
        return "Wear a raincoat, boots, and carry an umbrella."
    elif "cold" in weather:
        return "Wear a warm jacket, scarf, and gloves."
    else:
        return "Wear comfortable clothes suitable for mild weather."

# -----
# Build Agent
# -----
chat_model = ChatOpenAI(model="gpt-4o-mini", api_key=api_key)

system_prompt = SystemMessage(
    content="You are a travel assistant helping users with weather, route
planning, and clothing advice."
)

tools = [weather_api, route_api, clothing_api]

# Create the agent
agent = create_react_agent(chat_model, tools, prompt=system_prompt)

# -----
# Test Queries
# -----
queries = [
    "What is the weather like in Paris today?",
    "I am traveling to Tokyo tomorrow. Can you tell me the weather and what I
should wear?",
    "If I go to London today, what's the weather and what route should I
take?",
    "It's raining in Singapore. How should I travel to the Marina Bay area,
and what should I wear?",
    "I'm planning a trip to New York tomorrow. Please tell me the weather,
the best route to Central Park, and what clothing to pack."
]

for q in queries:
    print(f"\n--- Query: {q}")
    response = agent.invoke({"messages": [HumanMessage(q)]})
    for msg in response["messages"]:
        print(f"{msg.__class__.__name__}: {msg.content}")
```

 This way:

- Learners get **conceptual steps** (no copy-paste code).
- Instructor has a **complete runnable solution**.
- Both sides use the **same queries** so results can be compared directly.

Do you also want me to prepare a **worksheet/checklist** for the learners so they can record which tools were used for each query? That would make testing more structured.